

DIETLOG

Vision-Based Nutrition Tracking & Analytics Platform

Production-Grade Engineering & Architecture Design Specification

INSTITUTION COMSATS University Islamabad, Wah Campus	TARGET EVALUATION Semester Project Specifications (Industrial Standards)
DEPARTMENT Department of Computer Science (AI & Data Science Division)	ENGINEERING ARCHITECTURE Decoupled Microservice Architecture (Client-Server Blueprint)

1. Executive Summary & Problem Scope

Traditional dietary tracking implementations heavily suffer from extreme user friction due to manual inputs. Users are forced to interact with complex text queries, manually calculate portion sizes, and estimate macronutrient masses. This multi-step process introduces systemic inaccuracies and causes massive user churn.

DietLog addresses this direct bottleneck by transforming the user data ingestion pipeline into a single asynchronous vision-based capture stream. By deploying custom edge image capture pipelines and processing raw data payloads through state-of-the-art hosted Large Language Model (LLM) vision gateways (Gemini/Groq API), the system eliminates manual entry entirely. This document provides a complete production-grade blueprint for the application's implementation, detailing the database configuration, asynchronous network workflows, localized binary storage engines, directory patterns, and rigorous concurrent engineering workflows required for successful execution.

2. System Architecture & High-Level Pipeline

The DietLog network topology is strictly decoupled to ensure zero single points of failure and isolated scaling characteristics. The framework enforces an asynchronous, non-blocking single-request cycle across four distinct production layers.

Step 1: Client Ingestion Layer (Flutter UI Client)

Captures high-resolution raw image buffers using native camera abstractions. Encapsulates binary data payloads within a secure multipart/form-data HTTP client wrapper, appending localized cryptographic Bearer JWT access signatures within headers to prevent malicious traffic spikes.

Step 2: Processing & Guardrail Layer (Custom Application Server)

A self-managed FastAPI or Node.js application layer intercepting incoming streams. It validates client JWT identity, unserializes the binary image stream, enforces runtime payload scale limitations, and structures the external downstream communication pipelines.

Step 3: Structured Inference Layer (Gemini / Groq Gateway)

The application backend transfers the raw image tensor to the hosted inference model via secure API gateways. The request strictly forces a structural runtime schema constraint, compelling the LLM engine to map weights into a parsable mathematical JSON object while executing complete textual suppression.

Step 4: Atomic Persistence & Presentation Layer (PostgreSQL Engine)

The parsed structured response metrics are injected into a highly optimized, indexed PostgreSQL relational engine using raw parameterized SQL statements via dedicated connection pools. Upon write success, a JSON structural notification is returned to the client app to instantly update local state machines.

3. Machine Learning Inference & JSON Schema Constraint

To prevent un-parsable textual anomalies from downstream models, the system prompt strictly utilizes structural type injection. The downstream API is instructed to emit zero conversation or summary blocks. The only valid return entity is an un-nested, strict schema matching the exact architectural data specification below.

Inference Guardrail Rule: All structural outputs are checked programmatically inside backend interceptors before database persistence. Any deviation from primitive integer or float mappings throws an instantaneous HTTP 502 Bad Gateway exception.

```
{
  "food_item_identified": "String",
  "confidence_score": "Float",
  "calories_kcal": "Integer",
  "proteins_grams": "Float",
  "carbohydrates_grams": "Float",
  "fats_grams": "Float",
  "serving_size_estimate": "String"
}
```

4. Production Database Architecture (DDL Specifications)

The database backend completely bypasses resource-heavy Object-Relational Mappers (ORMs) to preserve sub-millisecond execution times. Database access is handled via direct connection pooling configurations utilizing

explicit thread allocation limits. Below is the complete relational DDL script containing composite indexing parameters optimized for historical queries across 7-day and 30-day temporal windows.

```
-- Enable Extension for Distributed Unique Identifier Generation
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- 1. Identity & Access Management Table
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Transactional Nutritional Logs Table
CREATE TABLE nutrition_logs (
    log_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    food_name VARCHAR(150) NOT NULL,
    calories INT NOT NULL CHECK (calories >= 0),
    protein_g NUMERIC(5,2) NOT NULL CHECK (protein_g >= 0),
    carbs_g NUMERIC(5,2) NOT NULL CHECK (carbs_g >= 0),
    fats_g NUMERIC(5,2) NOT NULL CHECK (fats_g >= 0),
    logged_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Create Composite Index to Accelerate Historical Aggregations and Dashboard Performance
CREATE INDEX idx_user_logs_date ON nutrition_logs (user_id, logged_at DESC);
```

5. Directory Engineering & Boilerplate Micro-Architecture

To preserve maximum codebase stability during concurrent multi-engineer feature deployment, the system architecture implements isolated monorepo top-level directories. This layout maintains clean dependency separation.

Frontend Architecture Directory (Flutter)

```
dietlog_frontend/  
├─ android/  
├─ ios/  
├─ lib/  
│   ├─ core/  
│   │   ├─ constants/  
│   │   └─ network/ (HTTP Client, JWT  
Interceptors)  
│   │   └─ utils/  
│   └─ data/  
│       ├─ local/ (Hive Boxes,  
SharedPreferences)  
│       └─ models/ (User & Log Serializers)  
│           └─ repositories/  
└─ logic/  
    └─ providers/ (Riverpod State  
Controllers)  
└─ presentation/  
    ├─ auth/ (Login/Signup Views)  
    └─ dashboard/ (Analytics Grid &  
Cards)  
        └─ history/ (ListView, DataTables)  
            └─ widgets/ (Re-usable Forms,  
Loaders)  
                └─ main.dart  
├─ pubspec.yaml  
└─ README.md
```

Backend Architecture Directory (FastAPI/Node)

```
dietlog_backend/  
├─ app/  
│   ├─ config/ (Database & Gemini  
Credentials)  
│   ├─ database/ (Connection Pool  
Initialization)  
│   └─ middlewares/ (JWT Security  
Validation)  
│       └─ models/ (Pydantic / Input Schemas)  
│           └─ routes/  
│               └─ auth.py (Manual JWT Signup/Login)  
│                   └─ logs.py (Image Multipart &  
Inference)  
└─ services/  
    └─ gemini_service.py (API  
Integration)  
└─ main.py  
├─ requirements.txt / package.json  
├─ .env  
└─ README.md
```

6. Rigorous Mapping of Academic & Production Curriculum Topics

The development map directly translates all production concepts from the academic tracking curriculum into operational system entities. The matrix below defines the exact component architecture constraints.

Curriculum Topic	Implementation Strategy inside DietLog App	Required Native Modules / Components
Dashboard Implementation	Acts as the initial high-impact workspace rendering user daily summaries.	Custom implementation using multi-axis layouts to host macro target metrics cleanly.
Bottom Navigation Bar	Facilitates smooth transitions between system application core paths.	Integrates standard persistent bars mapping views for Home, Camera capture, and Analytics.
ListView & Card View	Renders continuous historical logs chronologically for the user review.	Employs dynamic <code>ListView.builder</code> loops wrapped around customized elevation templates.
DataTable View	Presents detailed mathematical nutrient analytics for selected log records.	Compiles structured item matrices tracking accurate target metrics down to the exact gram.
Stack & Positioned Widgets	Designs responsive camera viewfinders and floating validation loaders.	Applies overlay containers to accurately lock design badges on top of real-time camera views.
Hive Local Database	Provides offline-first capabilities by storing analytics cached from the backend.	Opens encrypted local boxes to instantly cache metrics when remote servers are unreachable.
Shared Preferences	Maintains lightweight user parameters and session persistence state keys.	Saves local configuration parameters along with valid user access JWT keys on disk.
Riverpod Framework	Controls reactive global updates across the authentication and log engines.	Extends specialized state providers to trigger direct workspace updates on remote network success.

7. Concurrent Engineering & Git Branching Protocol

To prevent codebase corruption during parallel execution by the two-person engineering team, a strict branching standard is enforced. Direct pushes to the `main` branch are completely locked.

Day 1 Initialization Sequence (Developer A Tasks)

```
# Initialize Git Environment locally inside root development directory
git init

# Build remote reference map pointing to tracking platform repository
git remote add origin https://github.com/username/dietlog.git

# Generate isolated foundational project tracking environments
mkdir dietlog_frontend dietlog_backend
touch .gitignore README.md

# Commit and execute initialization push to upstream master server
git add .
git commit -m "chore: initial monorepo architecture skeleton"
git branch -M main
git push -u origin main
```

Concurrent Daily Execution Cycle (Developer A & Developer B Workflow)

To develop features simultaneously without overwriting teammate code blocks, both developers must run this exact script cycle every single working day:

```
# 1. Switch directly to personal assigned feature tracking path
git checkout feature/assigned-task-name

# 2. Extract upstream master adjustments and safely apply to local runtime space
git checkout main
git pull origin main
git checkout feature/assigned-task-name
git merge main

# 3. Execute core functional engineering additions... Verify performance metrics locally

# 4. Stage structural modifications and commit changes locally
git add .
git commit -m "feat: unique deterministic module functional specification descriptor"

# 5. Push local tracking branch changes cleanly to secure remote cloud repository
git push origin feature/assigned-task-name
```

Manual Conflict Resolution Protocol: When parallel edits on shared system assets trigger automated merge blocks, developers open target conflict lines via local standard IDE terminal inspectors, manually choose between Accept Current Change or Accept Incoming Change statements, save raw assets, and commit the final consolidated state safely.